Notas de la guía: Control de servomotores con Arduino

Fuente: How to Control Servo Motors with Arduino – Complete Guide (HowToMechatronics)
https://howtomechatronics.com/how-it-works/how-servo-motors-work-how-to-control-servos-using-arduino/

---

a) Principio de funcionamiento de un servomotor

Un servomotor es un sistema de lazo cerrado. Mide su propia posición y la corrige hasta llegar al ángulo deseado. Un servo de hobby tiene cuatro componentes internos: un motor DC, una caja de engranajes, un potenciómetro y un circuito de control. El motor DC gira rápido pero con poco torque. Los engranajes reducen la velocidad a unos 60 RPM y aumentan la fuerza.

El potenciómetro está unido al eje de salida y entrega un voltaje proporcional al ángulo real. El circuito de control compara ese voltaje con la señal que recibe. Si hay diferencia, activa un puente H interno que gira el motor en el sentido necesario hasta que el error sea cero. En los servos industriales, en lugar de potenciómetro se usa un encoder de alta precisión.

La señal de control es un tren de pulsos a 50 Hz, es decir, un pulso cada 20 ms. El ancho del pulso determina el ángulo:

Ancho del pulso	Ángulo
1 ms	0°
1.5 ms	90°
2 ms	180°

Según la marca, estos valores pueden ir de 0.5 ms a 2.5 ms. Los servos estándar giran unos 180° porque tienen topes físicos. Los de 360° (rotación continua) no permiten controlar la posición, solo la velocidad y el sentido de giro.

b) Conexiones básicas (alimentación, tierra y señal)

El servo tiene solo tres cables:

- **Marrón** → tierra (GND).
- **Rojo** (el del medio) → alimentación positiva (5 V).
- **Amarillo o naranja** → señal de control, que va a cualquier pin digital del Arduino.

Si el servo usa una fuente externa, la tierra del Arduino **debe unirse** a la tierra del servo. Sin esa referencia común, la señal no se interpreta correctamente.

La guía compara dos modelos populares:

- **SG90 (micro servo):** pesa 9 g y trabaja de 3.5 a 6 V. Consume unos 100–200 mA en movimiento y hasta unos 650 mA bloqueado.
- **MG996R:** tiene engranajes metálicos, alto torque (unos 10–13 kg·cm) y trabaja de 4.8 a 7.2 V. Consume de 500 a 900 mA en funcionamiento y hasta 2.5 A bloqueado. Por eso no se debe conectar directamente al Arduino.

c) Control mediante programación en Arduino

La guía muestra primero cómo generar la señal “a mano”, sin librería. Se pone el pin en HIGH durante el ancho del pulso y en LOW el resto de los 20 ms. Por ejemplo, para 90° con un pulso de 1450 µs, el pin queda 1450 µs en alto y

18 550 µs en bajo (20 000 − 1450).

Al probar su servo, el autor encontró estos valores reales:

Ancho del pulso	Ángulo
0.6 ms	0°
1.45 ms	90°
2.3 ms	180°

Esto muestra que cada servo debe calibrarse.

```
void loop() {
  digitalWrite(9, HIGH);
  delayMicroseconds(1450);   // ancho del pulso -> ~90°
  digitalWrite(9, LOW);
  delayMicroseconds(18550);  // resto del periodo de 20 ms
}
```

d) Uso de la librería Servo

La forma más práctica es usar la librería `Servo.h`. Los pasos son:

- Incluir la librería y crear un objeto `Servo`.
- Usar `attach(pin, min, max)` para indicar el pin y, opcionalmente, los anchos de pulso mínimo y máximo en microsegundos. Por defecto son 544 µs para 0° y 2400 µs para 180°.
- Usar `write(ángulo)` para mover el servo a cualquier posición entre 0° y 180°.

Si el servo no recorre todo el rango de 0 a 180°, se corrige ajustando esos valores min/max en `attach()`.

```
#include <Servo.h>
Servo miServo;
void setup() { miServo.attach(9, 600, 2300); }
void loop() {
  miServo.write(0);   delay(1000);
  miServo.write(90);  delay(1000);
  miServo.write(180); delay(1000);
}
```

e) Consideraciones para la alimentación de múltiples servomotores

El pin de 5 V del Arduino entrega solo unos 500 mA si se alimenta por USB, o hasta 1 A con el conector de alimentación. Un SG90 podría funcionar directo, pero el autor recomienda **siempre usar una fuente externa** para mayor estabilidad. Con varios servos o con el MG996R, la fuente externa es obligatoria. La guía usa una de 5 V a 2 A para un servo y de 5 V a 6 A para varios. El amperaje alto no daña nada: solo indica cuántos servos puede mover a la vez.

Un problema común es que el servo tiembla o reinicia el Arduino por los picos de corriente al arrancar. La solución es colocar un **capacitor de desacople entre GND y 5 V**, que aporta corriente extra en esos picos.

En cuanto a cantidad, la librería Servo controla:

- Hasta **12 servos** en la mayoría de las placas.
- Hasta **48 servos** en el Arduino Mega. El autor controló 22 servos en su robot hexápodo.

Para muchos servos, la guía presenta el driver **PCA9685**:

- Tiene 16 canales PWM de 12 bits (4096 pasos).
- Se comunica por I2C con solo dos pines.
- Tiene reloj propio, así que mueve los servos sin cargar al Arduino.
- Se pueden encadenar hasta 62 drivers, lo que en teoría permite 992 servos.
- Cada driver necesita su propia dirección (configurada con pads de soldadura) y también requiere fuente separada para los servos.

f) Ejemplos de movimiento angular y control de posición

La guía incluye varios ejemplos:

- **Movimiento por posiciones fijas:** un servo pasa por 0°, 90°, 135° y 180° con pausas entre cada posición mediante `delay()`.
- **Varios servos con la librería:** se crea un objeto por servo (por ejemplo, en los pines 8 a 12) y se mueven todos juntos a 0°, luego a 90° y luego a 180°.
- **Con el PCA9685:** se configura la frecuencia a 50 Hz y se usan las funciones `setChannelPWM()` y `pwmForAngle()` para llevar cada canal al ángulo deseado. Como cada servo responde un poco distinto, se calibra individualmente, incluso ajustando su punto de 90°.

Las aplicaciones mencionadas incluyen brazos robóticos, robots hexápodos, robots SCARA y modelos RC.